«controlador»
Controladores Flask
app/controlador

publico.bp: inicio, ingresar, registro, salir
aspirante.bp: cuestionario, evaluar, evaluacion, inicio
ingeniero.bp: panel, granulos, granulo_*, reglas, evaluaciones

usa

«fachada del modelo»
SistemaExperto
app/modelo/sistema_experto.py

+aspirantes: AspiranteDAO
+ingenieros: IngenieroDAO
+preguntas: PreguntaDAO
+hechos: HechoDAO +hipotesis: HipotesisDAO
+granulos: GranuloDAO
+reglas: ReglaNoConcluyenteDAO
+evaluaciones: EvaluacionDAO
+motor: MotorInferencia

+registrar_aspirante(datos, contrasena): Aspirante
+autenticar(rol, correo, contrasena)
+evaluar(id_aspirante, textos): int
+crear_granulo(id_ing, id_hecho, id_hip, peso): str
+actualizar_granulo(id_ing, id_granulo, peso, activo)
+resumen(): dict

1

«servicio»
MotorInferencia
motor_inferencia.py

-_hechos: HechoDAO
-_granulos: GranuloDAO
-_reglas: ReglaNoConcluyenteDAO

+inferir(respuestas): ResultadoInferencia

crea

TablaContabilizacion

-_filas: dict<id_hipotesis, ConteoHipotesis>
-_maximos: dict

+registrar(granulo, hecho) (peso × grado)
+calcular(): list<ConteoHipotesis>

consulta

1 instancia de cada DAO (comparten la conexión de la petición)

paquete app.modelo.dao

AspiranteDAO
transaccional.aspirantes

+insertar(documento, nombres, apellidos, correo, contrasena_hash): Aspirante
+credenciales(correo): tuple
+contar(): int

IngenieroDAO
conocimiento.ingenieros_conocimiento

+credenciales(correo): tuple

PreguntaDAO
conocimiento.preguntas_cuestionario_...

+activas(): list<Pregunta>

HipotesisDAO
conocimiento.hipotesis_programas

+activas(): list

«abstract»
BaseDAO
base_dao.py

#_conexion: callable (conexion_actual)

#_consultar(sql, parametros): list
#_consultar_uno(sql, parametros): dict | None
#_transaccion(): cursor (COMMIT / ROLLBACK)

HechoDAO
conocimiento.hechos_perfil (RC)

+derivar(respuestas): (hechos, sin_hecho)
+listar(): list
+rangos_por_variable(): dict

GranuloDAO
conocimiento.granulos_hecho_hipotesis (RG)

+por_hecho(id_hecho): list<Granulo>
+maximos_por_hipotesis(): dict
+listar(codigo_hipotesis): list
+obtener(id_granulo): dict
+resumen_por_hipotesis(): list
+contar_activos(): int
+insertar(id_hecho, id_hip, peso, id_ing): str
+actualizar(id_granulo, peso, activo, id_ing)

ReglaNoConcluyenteDAO
conocimiento.reglas_resultado_no_... (RN)

+activas(): list<ReglaNoConcluyente>

EvaluacionDAO
transaccional: 6 tablas de la evaluación

+insertar(id_asp, respuestas, resultado): int
+listar_por_aspirante(id_aspirante): list
+listar_todas(texto, limite): list
+detalle(id_evaluacion): dict
+contar(): int
+programas_mas_frecuentes(limite): list

usa la conexión

FabricaConexion
app/modelo/conexion.py

-_pool: ThreadedConnectionPool (psycopg2)

+obtener(): connection
+devolver(conexion)
+verificar()
+cerrar()

SQL

PostgreSQL
sistema_experto_orientacion
esquemas: conocimiento y transaccional

devuelve

«dataclass» entidades.py — objetos que devuelven los DAO y el motor:

Aspirante | IngenieroConocimiento | Pregunta | Respuesta | Hecho | Granulo | ReglaNoConcluyente | ConteoHipotesis | ResultadoInferencia